TALLER 2 : IMPLEMENTACION APLICACIÓN DOCKER

- INTEGRANTES : - ELEANYERIS HERNANDEZ , GISEELLA SANCHEZ , ANGELINA OCHOA

13 DE OCTUBRE DEL 2026

INSTRUCTOR : OMAR GUTIERREZ

SENA

ADSO-17

BARRANQUILLA- ATLÁNTICO

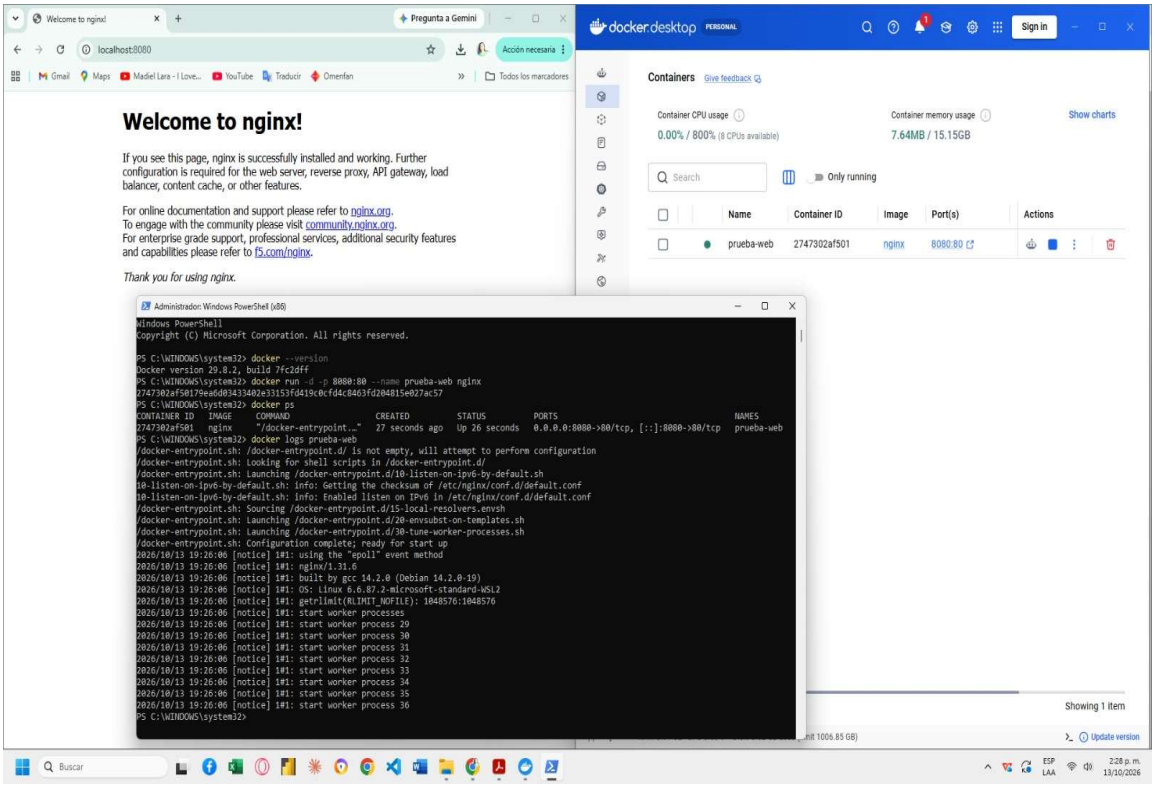
Enlace del Repositorio:

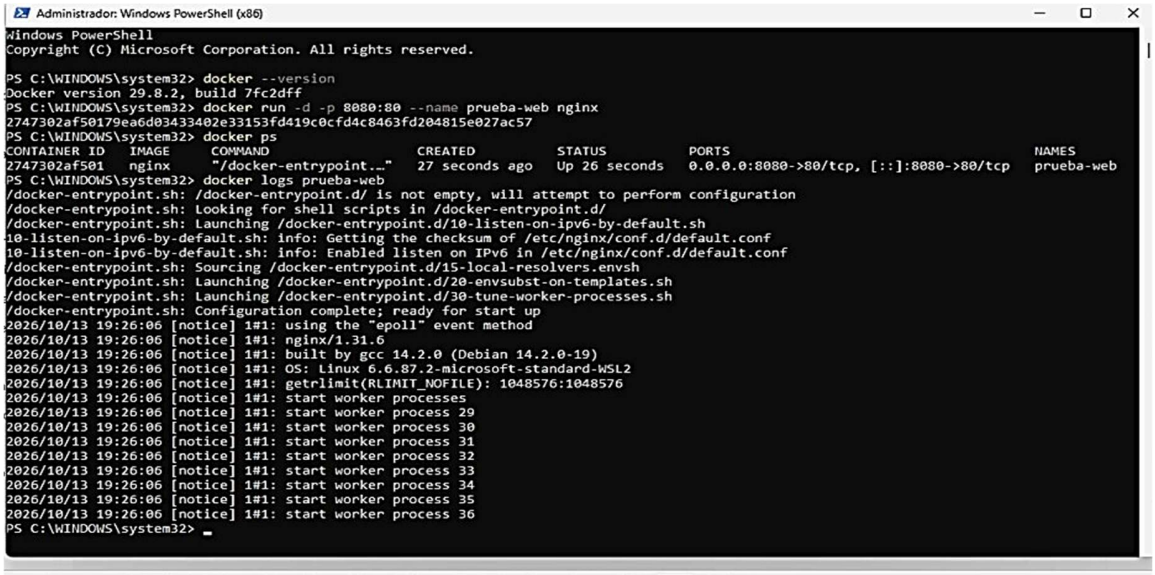
https://github.com/gpsanchezr/fullstack-docker-task-manager.git

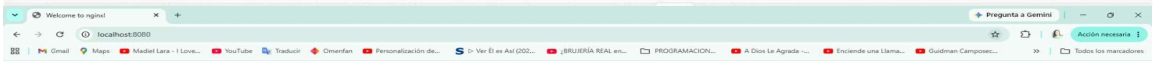
A continuación, estarán las capturas como evidencia de nuestro trabajo:

📸 Evidencia 1 (Fase 1: Calentamiento)

- **Cuándo la tomamos:** En el Paso 7 de la Fase 1.
- **Comandos previos:** Ejecutamos docker run -d -p 8080:80 --name prueba-web nginx y luego docker ps.
- **Qué se ve en nuestra captura:** Nuestra consola PowerShell mostrando una fila con el contenedor llamado prueba-web y debajo de la columna STATUS se evidencia el estado Up.







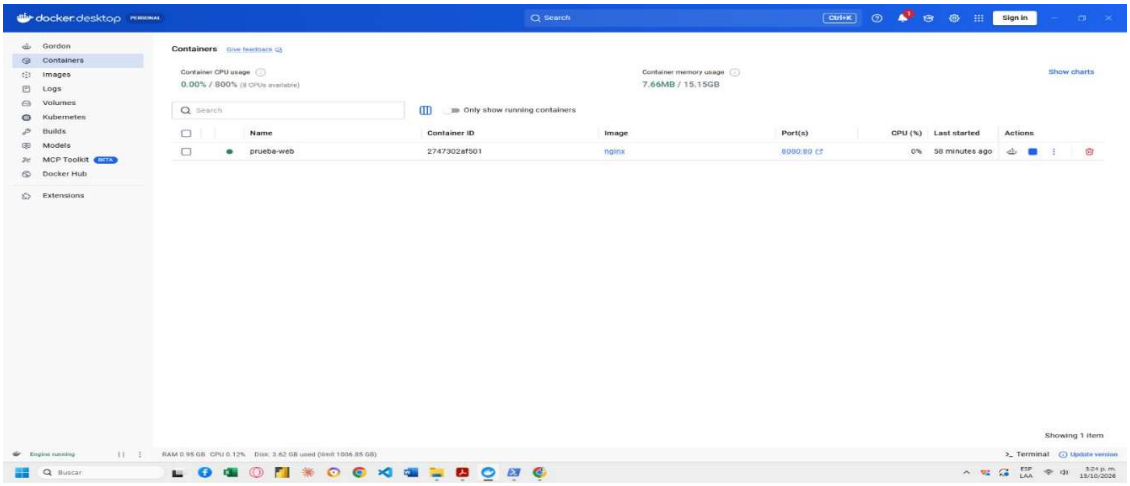
Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

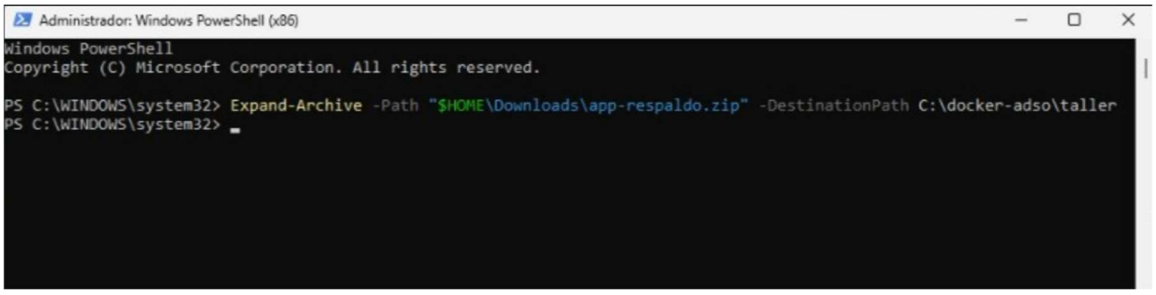
Thank you for using nginx.



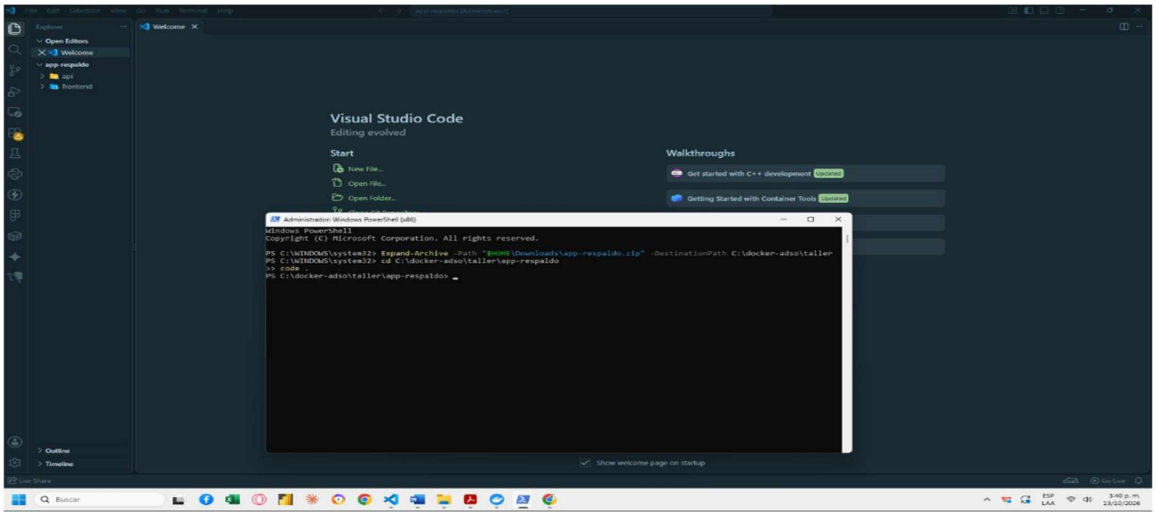


📸 Evidencia 2 (Fase 2: Preparar la API)

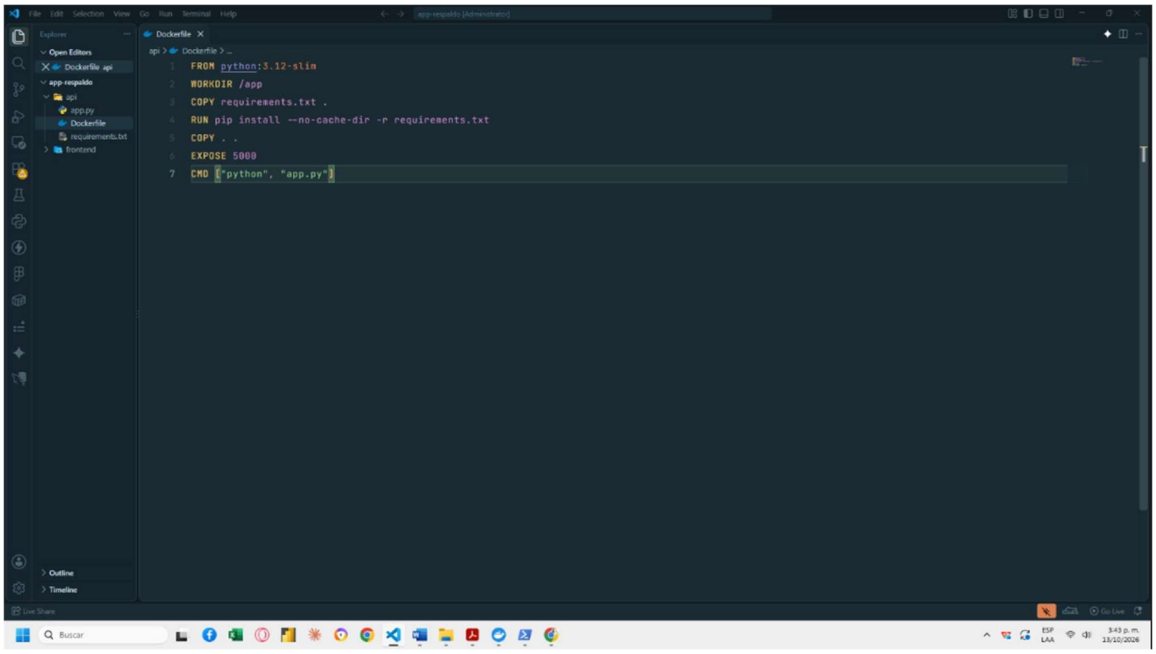
- **Cuándo la tomamos:** En el Paso 11 de la Fase 2.
- **Comandos previos:** Construimos la imagen con docker build -t tareas-api:1.0 ., la encendimos con docker run ... tareas-api:1.0 y revisamos sus registros con docker logs api-prueba.
- **Qué se ve en nuestra captura (son dos cosas):**
    1. El final del proceso de construcción donde dice naming to docker.io/library/tareas-api:1.0.
    2. El resultado de los logs indicando Running on http://0.0.0.0:5000, demostrando que arrancó correctamente.



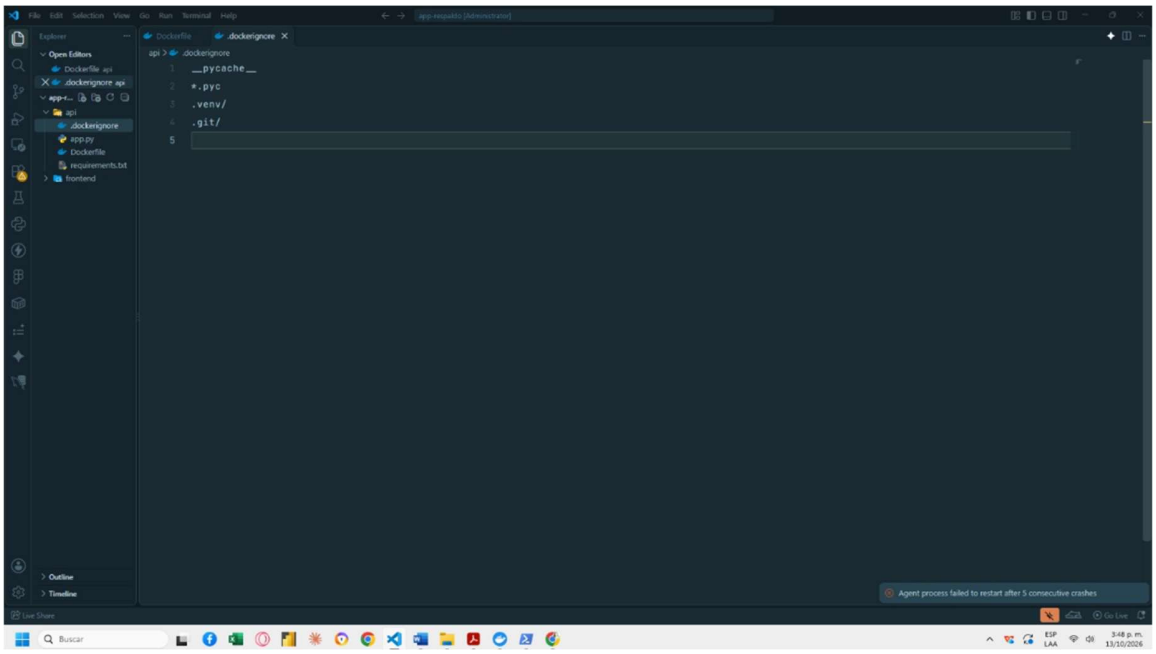
Se entra a la carpeta y se abre VS Code:



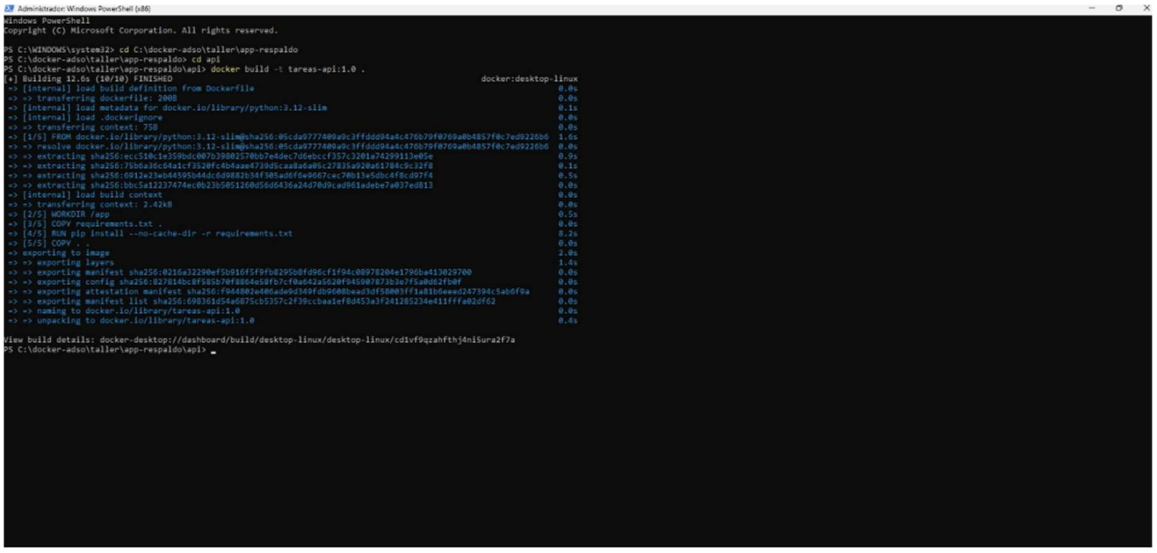
Creo el Dockerfile: En VS Code, hago clic derecho sobre la carpeta api, selecciono "Nuevo archivo", lo nombro exactamente Dockerfile:



Creo el archivo .dockerignore en la misma carpeta 📁 api:

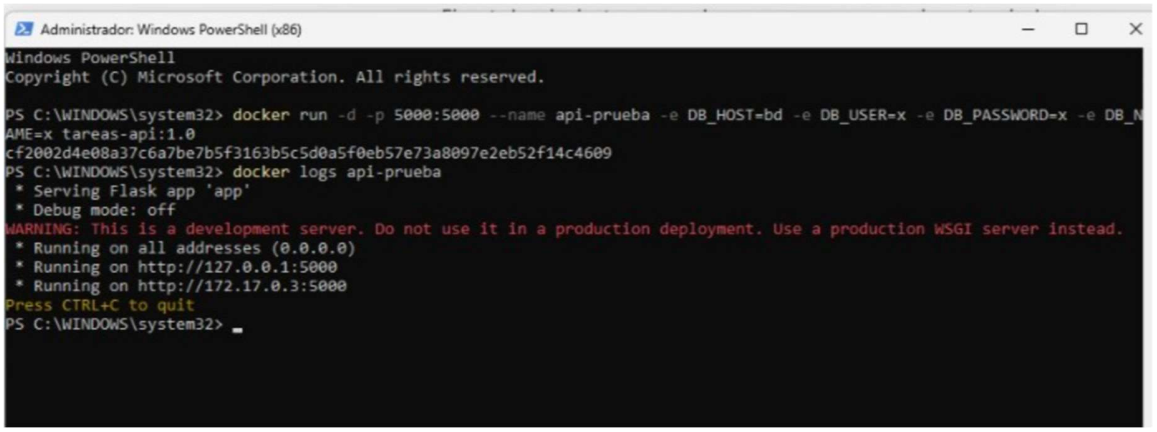


Construyo la imagen: Abro una nueva terminal en VS Code (Terminal > Nueva terminal), entro a la carpeta de la API y ejecuto el comando para iniciar la construcción:



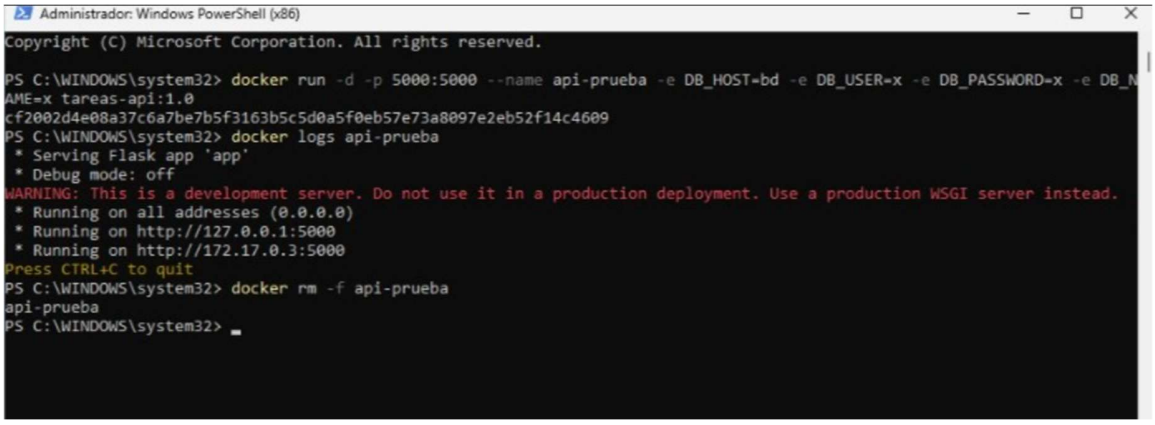
Verifico el éxito de la construcción: La creación de la imagen es un éxito total, lo cual confirmo al ver el mensaje naming to docker.io/library/tareas-api:1.0 al final de mi consola.

Evidencia 2: Se enciende el contenedor de prueba y se revisa los registros para ver si arrancó:



Borro el contenedor de prueba: Ahora, antes de continuar, me aseguro de ejecutar este comando en esa misma consola de PowerShell para eliminar el contenedor de prueba y liberar el puerto 5000: en powershell colocamos:

docker rm -f api-prueba



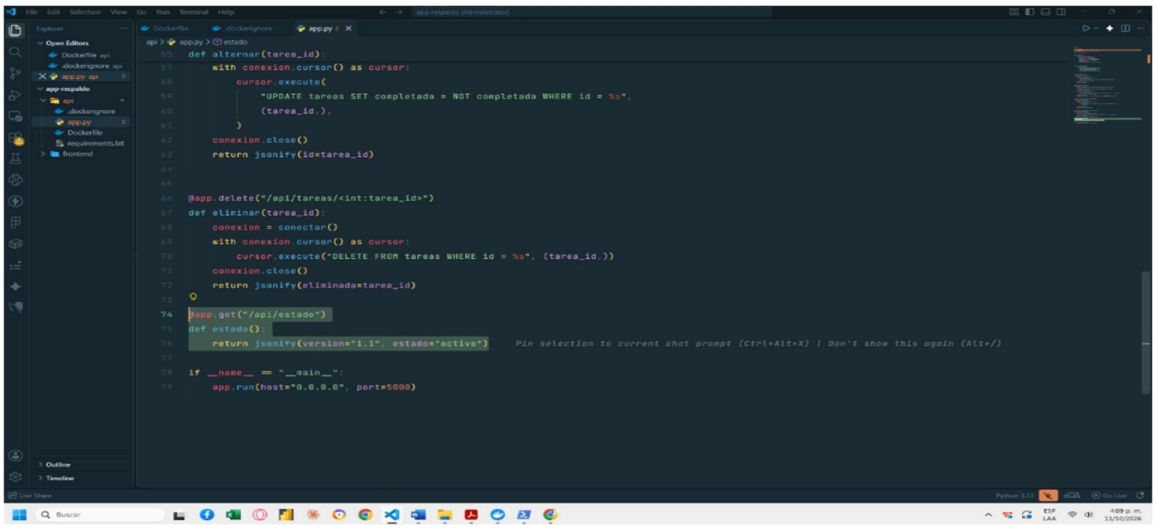
Evidencia 3 (Fase 3: Versión 1.1)

- **Cuándo la tomamos:** En el Paso 6 de la Fase 3.
- **Comandos previos:** Construimos la versión 1.1, encendimos ambos contenedores (api-v10 y api-v11) y ejecutamos docker images tareas-api.
- **Qué se ve en nuestra captura (son tres cosas):**
    1. Nuestra terminal de VS Code mostrando las dos versiones de la imagen (la que tiene etiqueta 1.0 y la 1.1).
    2. Una ventana de nuestro navegador en http://localhost:5001/api/estado mostrando el error Not Found.
    3. Otra ventana de nuestro navegador en http://localhost:5002/api/estado mostrando el mensaje {"estado": "activo", "version": "1.1"}.

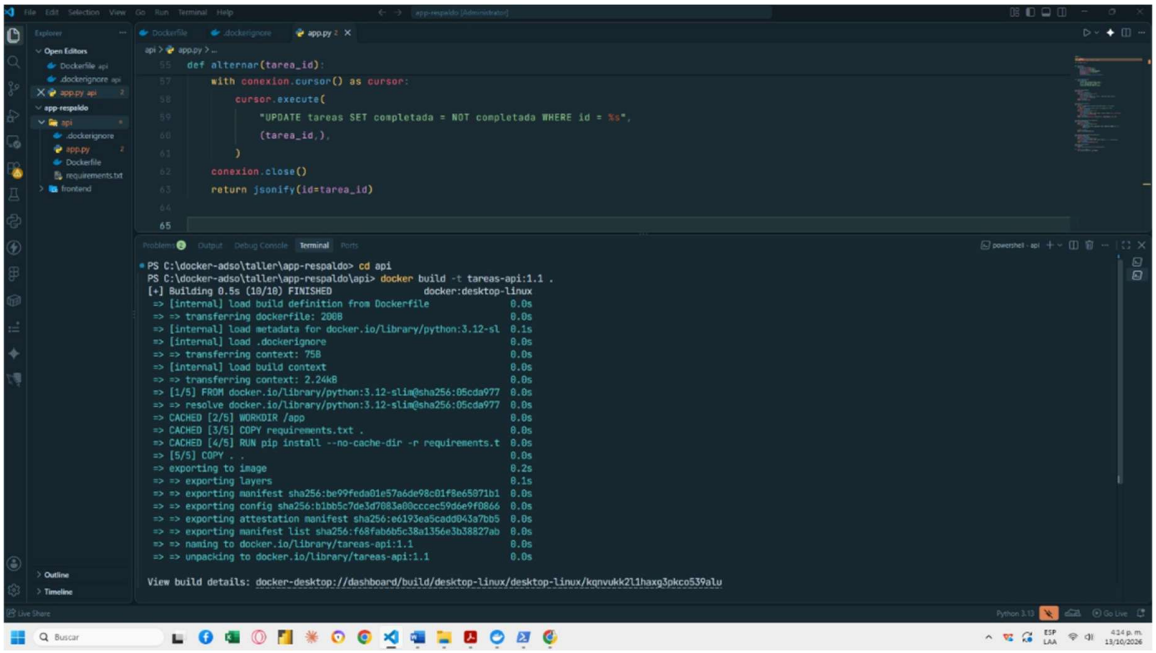
Agregamos al app.apy: @app.get("/api/estado")

def estado():

return jsonify(version="1.1", estado="activo")

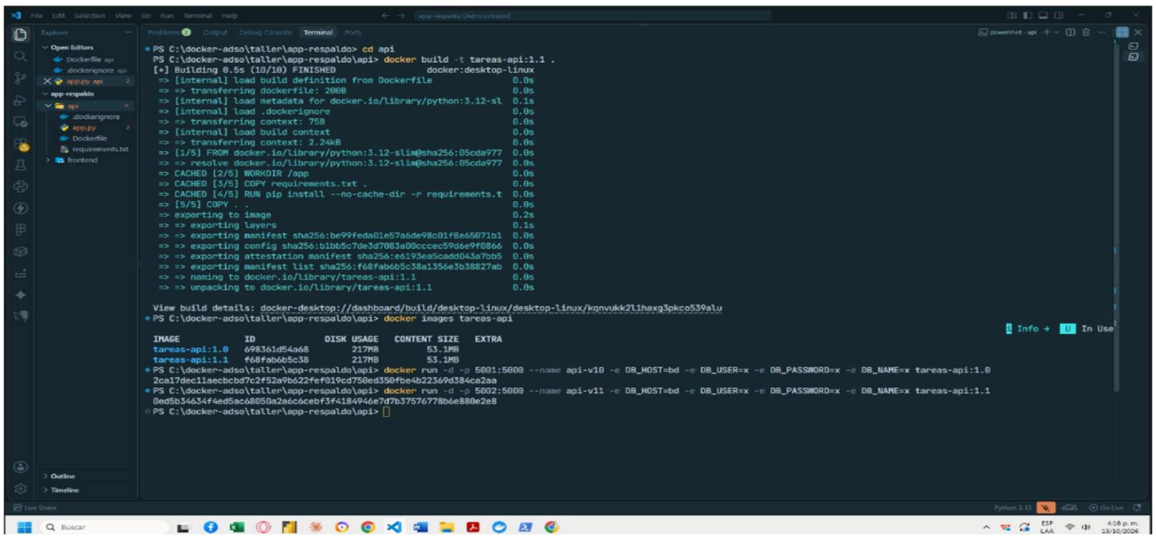


Construyo la versión 1.1: Abro mi terminal de PowerShell y me aseguro de estar situado exactamente en la ruta C:\docker-adso\taller\app-respaldo\api>.



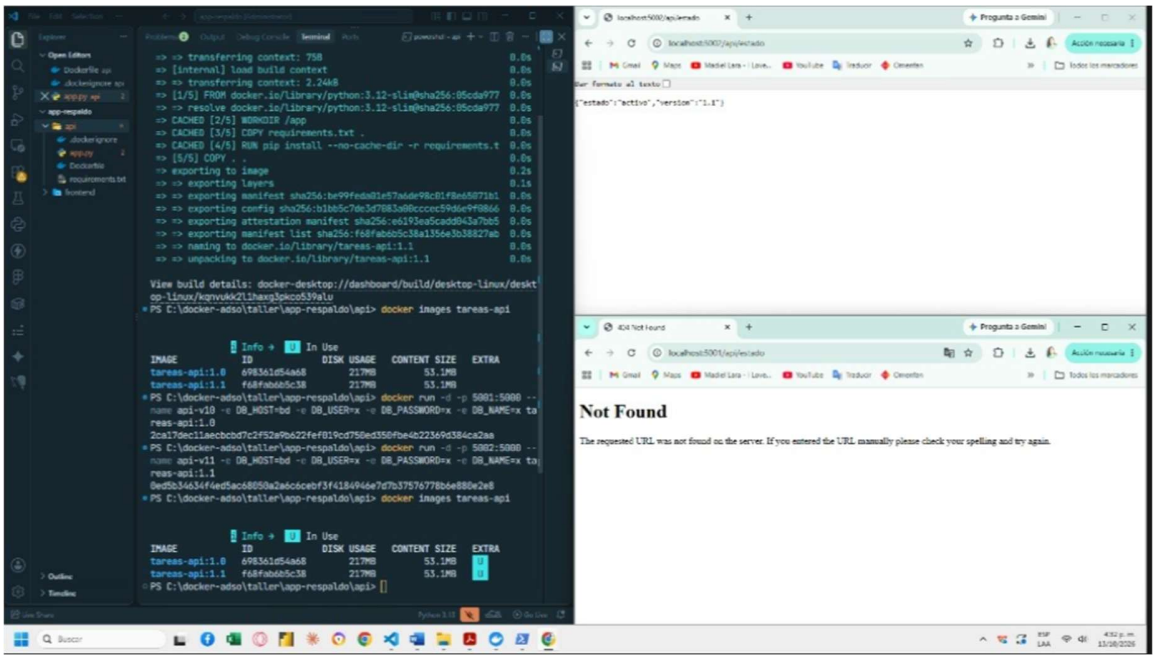
Verificación y encendido de nuestras versiones:

En esta captura verificamos con el comando docker images que las dos versiones de nuestra API (1.0 y 1.1) se construyeron correctamente. Luego, procedimos a encender ambas versiones al mismo tiempo, asignando el puerto 5001 a la versión 1.0 y el puerto 5002 a la nueva versión 1.1, para poder evaluarlas de forma simultánea.

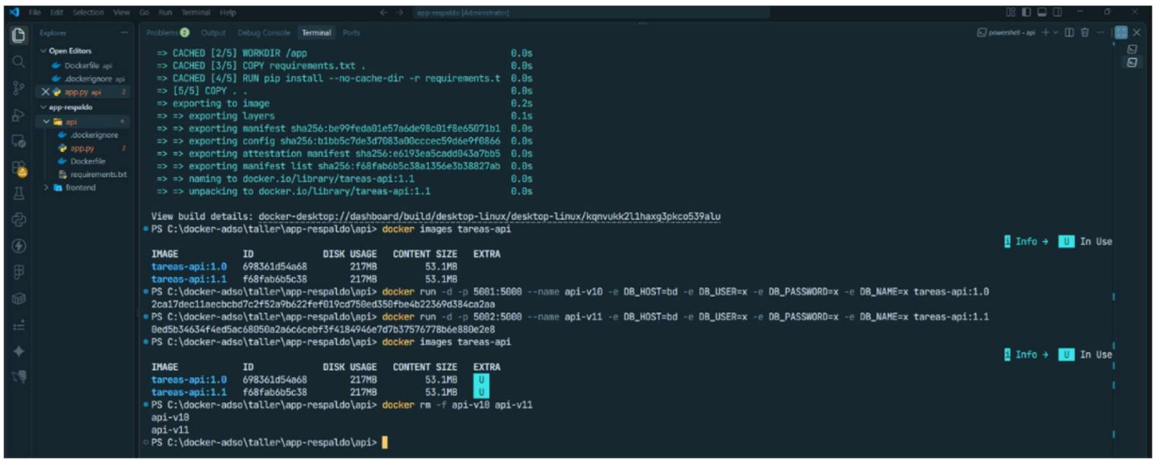


El puerto 5001 corre la Versión 1.0: Esta primera versión de la imagen la construimos antes de que escribiéramos la nueva ruta @app.get("/api/estado") en el código de Python. Por lo tanto, cuando intentamos buscar /api/estado en el navegador usando esta versión vieja, nos responde con un error Not Found porque esa ruta literalmente no existe allí.

El puerto 5002 corre la Versión 1.1: Esta versión la construimos después de que guardamos el código nuevo. Por eso, al entrar a este puerto, sí nos devuelve correctamente el JSON {"estado": "activo", "version": "1.1"}.



limpio la consola con docker rm -f api-v10 api-v11 y borras esas pruebas:



Evidencia 4 (Fase 4: Persistencia y Volúmenes)

- **Cuándo la tomamos:** En el Paso 8 de la Fase 4.
- **Comandos previos:** Eliminamos nuestra primera base de datos, creamos una nueva (bd-taller) conectada al mismo volumen datos-taller, y ejecutamos el comando de consulta docker exec -it bd-taller mysql ... -e "SELECT * FROM prueba;".
- **Qué se ve en nuestra captura:** Nuestra consola de VS Code mostrando el resultado de la consulta SQL con el registro 1 | Registro persistente, demostrando que los datos sobrevivieron aunque borramos el contenedor original.

Voy a encender MySQL con un volumen. Para lograrlo, crearé un contenedor llamado bd-taller y le asignaré un volumen llamado datos-taller.

1. Encender MySQL con un volumen persistente: Creo el contenedor bd-taller vinculando el volumen datos-taller para asegurar que la informacion no se pierda.

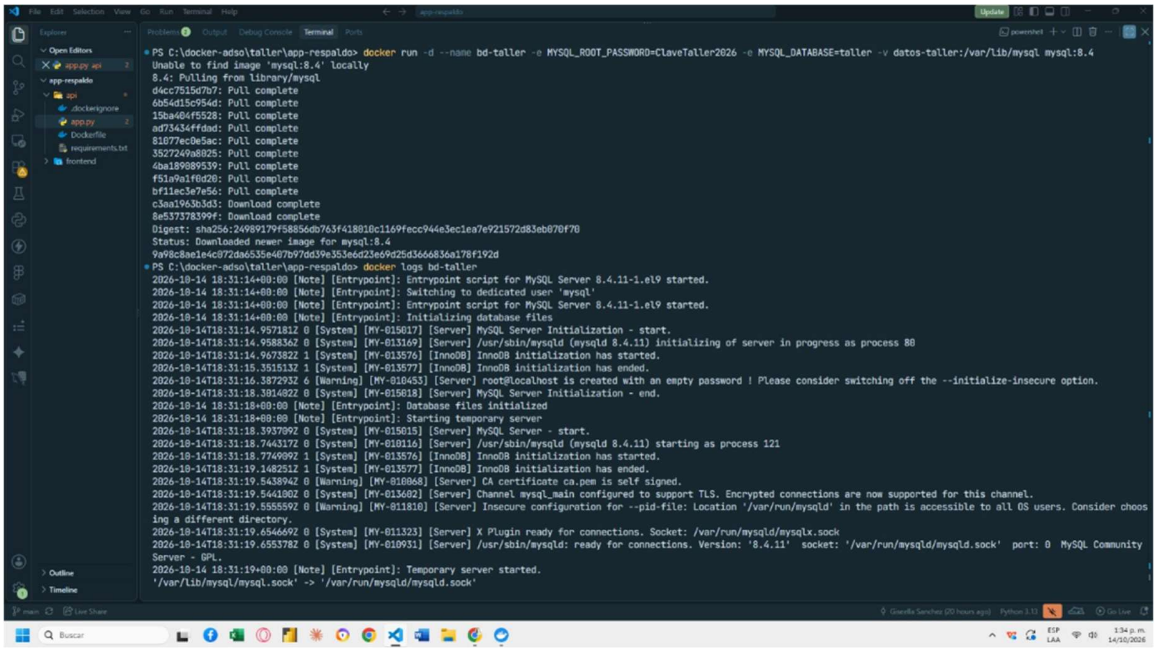
2. Verificar que el servicio arranco: Espero unos segundos y reviso los logs del contenedor para confirmar que la base de datos esta lista y aceptando conexiones.

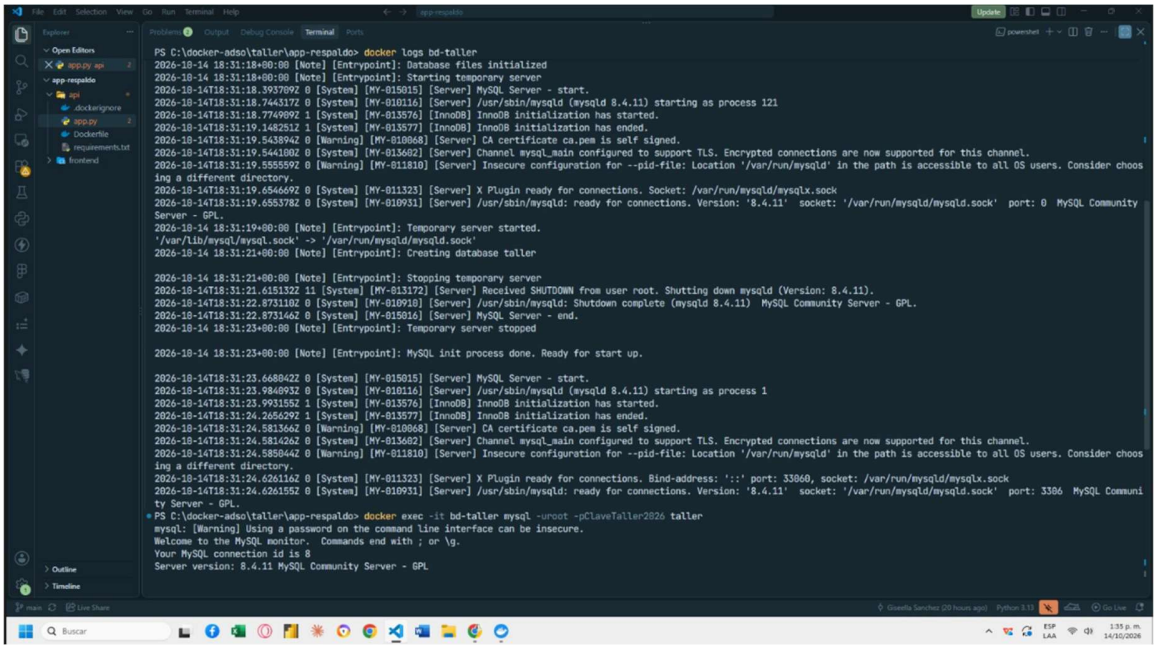
3. Entrar a la base de datos: Me conecto al contenedor de MySQL de forma interactiva para poder ejecutar comandos directamente en su consola.

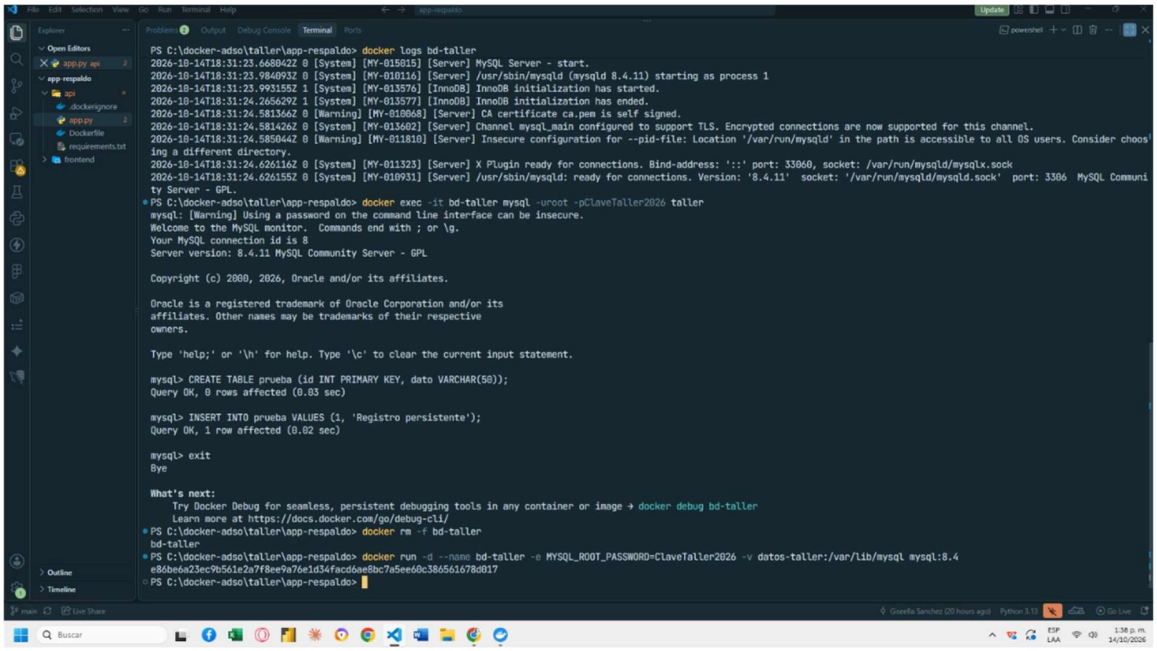
4. Crear una tabla de prueba y registrar un dato: Escribo y ejecuto las sentencias SQL necesarias dentro de la consola para generar la estructura inicial e insertar un registro de prueba.

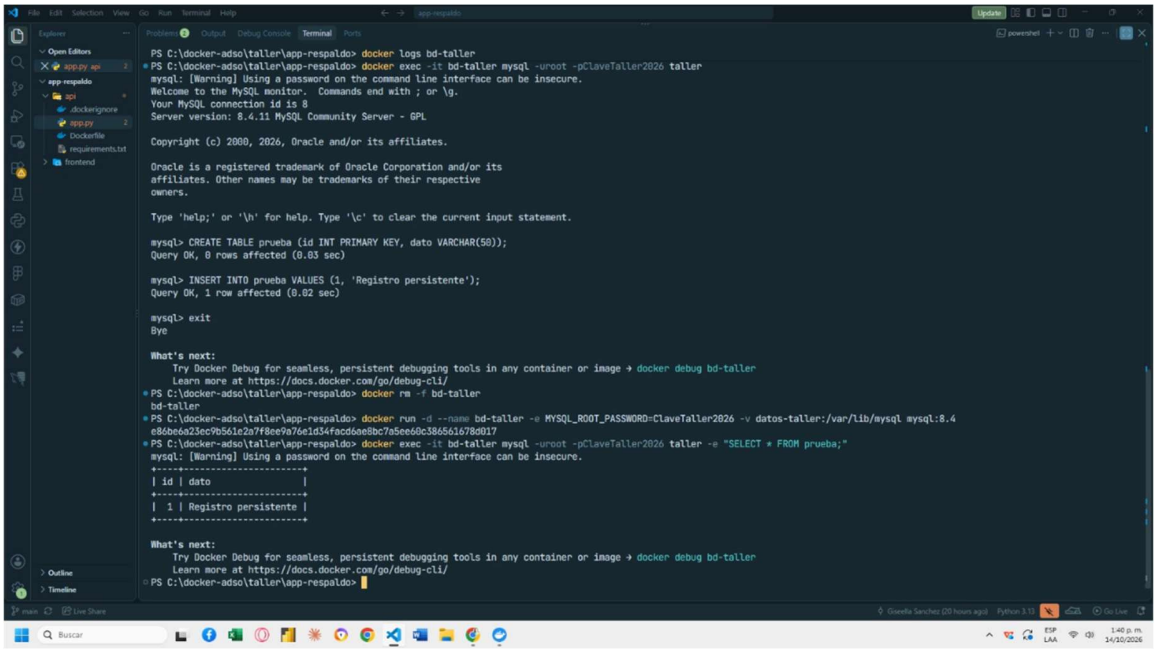
5. Probar la persistencia eliminando y recreando el contenedor: Elimino el contenedor actual para demostrar que los datos sobreviven gracias al volumen, y posteriormente lo vuelvo a levantar para verificar la información.

6. Verificar la persistencia: Consulto los datos para comprobar que la informacion sigue ahi a pesar de haber borrado el contenedor anterior.

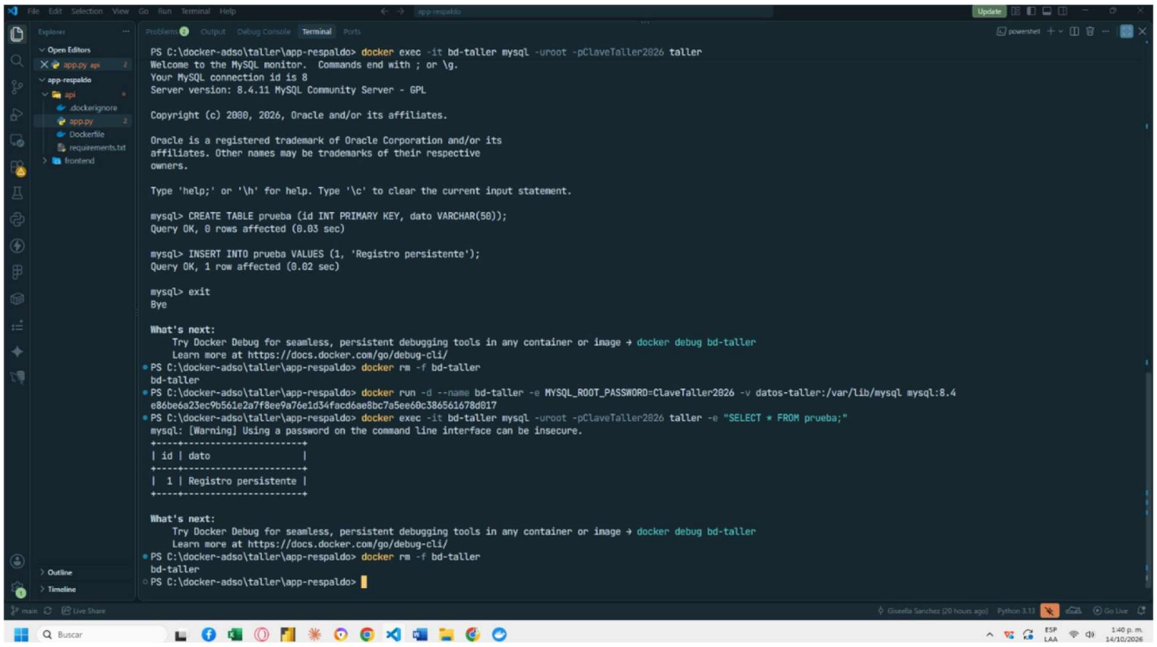

PS C:\docker-adso\taller\app-respaldo> docker run -d --name bd-taller -e MYSQL_ROOT_PASSWORD=ClaveTaller2026 -e MYSQL_DATABASE=taller -v datos-taller:/var/lib/mysql mysql:8.4
Unable to find image 'mysql:8.4' locally
8.4: Pulling from library/mysql
d4cc7515d7b7: Pull complete
6b564d15c954d: Pull complete
15ba404f5528: Pull complete
ad73434ffdad: Pull complete
81077ec0e5ac: Pull complete
3527249a8825: Pull complete
4ba189089539: Pull complete
f51a9a1f0d20: Pull complete
bf11ec3e7e56: Pull complete
c3aa1963b3d3: Download complete
8e537378399f: Download complete
Digest: sha256:249891179f58856eb763f418018c1169fecc944e3ec1ea7e921572d83eb070f70
Status: Downloaded newer image for mysql:8.4
9a98c8ae1e4c872da6535e407b97dd39e353e6d23e69d25d3666836a178f192d
PS C:\docker-adso\taller\app-respaldo> docker logs bd-taller
2026-10-14 18:31:14+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.11-1.el9 started.
2026-10-14 18:31:14+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
2026-10-14 18:31:14+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.11-1.el9 started.
2026-10-14 18:31:14+00:00 [Note] [Entrypoint]: Initializing database files
2026-10-14T18:31:14.957181Z 0 [System] [MY-015017] [Server] MySQL Server Initialization - start.
2026-10-14T18:31:14.958836Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.4.11) initializing of server in progress as process 80
2026-10-14T18:31:14.967382Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-10-14T18:31:15.351513Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-10-14T18:31:16.387293Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an empty password ! Please consider switching off the --initialize-insecure option.
2026-10-14T18:31:18.301402Z 0 [System] [MY-015018] [Server] MySQL Server Initialization - end.
2026-10-14 18:31:18+00:00 [Note] [Entrypoint]: Database files initialized
2026-10-14 18:31:18+00:00 [Note] [Entrypoint]: Starting temporary server
2026-10-14T18:31:18.393709Z 0 [System] [MY-015015] [Server] MySQL Server - start.
2026-10-14T18:31:18.744317Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.4.11) starting as process 121
2026-10-14T18:31:18.774909Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-10-14T18:31:19.148251Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-10-14T18:31:19.543894Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2026-10-14T18:31:19.544100Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel.
2026-10-14T18:31:19.555559Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choos
ing a different directory.
2026-10-14T18:31:19.654669Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Socket: /var/run/mysqld/mysqlx.sock
2026-10-14T18:31:19.655378Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.4.11' socket: '/var/run/mysqld/mysqld.sock' port: 0 MySQL Community
Server - GPL.
2026-10-14 18:31:19+00:00 [Note] [Entrypoint]: Temporary server started.
'/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'


PS C:\docker-adso\taller\app-respaldo> docker logs bd-taller
2026-10-14 18:31:18+00:00 [Note] [Entrypoint]: Database files initialized
2026-10-14 18:31:18+00:00 [Note] [Entrypoint]: Starting temporary server
2026-10-14T18:31:18.393709Z 0 [System] [MY-015015] [Server] MySQL Server - start.
2026-10-14T18:31:18.744317Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.4.11) starting as process 121
2026-10-14T18:31:18.774909Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-10-14T18:31:19.148251Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-10-14T18:31:19.543894Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2026-10-14T18:31:19.544100Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel.
2026-10-14T18:31:19.555559Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choos
ing a different directory.
2026-10-14T18:31:19.654669Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Socket: /var/run/mysqld/mysqlx.sock
2026-10-14T18:31:19.655378Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.4.11' socket: '/var/run/mysqld/mysqld.sock' port: 0 MySQL Community
Server - GPL.
2026-10-14 18:31:19+00:00 [Note] [Entrypoint]: Temporary server started.
'/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
2026-10-14 18:31:21+00:00 [Note] [Entrypoint]: Creating database taller
2026-10-14 18:31:21+00:00 [Note] [Entrypoint]: Stopping temporary server
2026-10-14T18:31:21.615132Z 11 [System] [MY-013172] [Server] Received SHUTDOWN from user root. Shutting down mysqld (Version: 8.4.11).
2026-10-14T18:31:22.873110Z 0 [System] [MY-010910] [Server] /usr/sbin/mysqld: Shutdown complete (mysqld 8.4.11) MySQL Community Server - GPL.
2026-10-14T18:31:22.873146Z 0 [System] [MY-015016] [Server] MySQL Server - end.
2026-10-14 18:31:23+00:00 [Note] [Entrypoint]: Temporary server stopped
2026-10-14 18:31:23+00:00 [Note] [Entrypoint]: MySQL init process done. Ready for start up.
2026-10-14T18:31:23.668042Z 0 [System] [MY-015015] [Server] MySQL Server - start.
2026-10-14T18:31:23.984093Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.4.11) starting as process 1
2026-10-14T18:31:23.991155Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-10-14T18:31:24.265629Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-10-14T18:31:24.581366Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
2026-10-14T18:31:24.581426Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel.
2026-10-14T18:31:24.585044Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choos
ing a different directory.
2026-10-14T18:31:24.626116Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port: 33060, socket: /var/run/mysqld/mysqlx.sock
2026-10-14T18:31:24.626155Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.4.11' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Communi
ty Server - GPL.
PS C:\docker-adso\taller\app-respaldo> docker exec -it bd-taller mysql -uroot -pClaveTaller2026 taller
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.11 MySQL Community Server - GPL



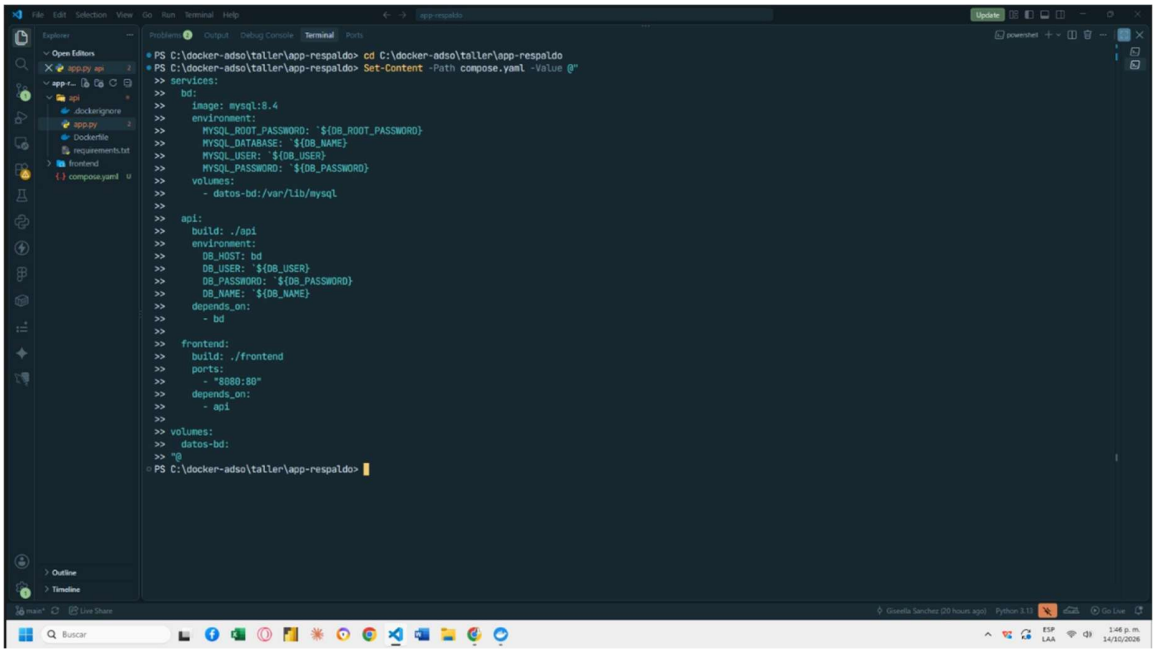


Limpiamos Contenedor de Prueba:

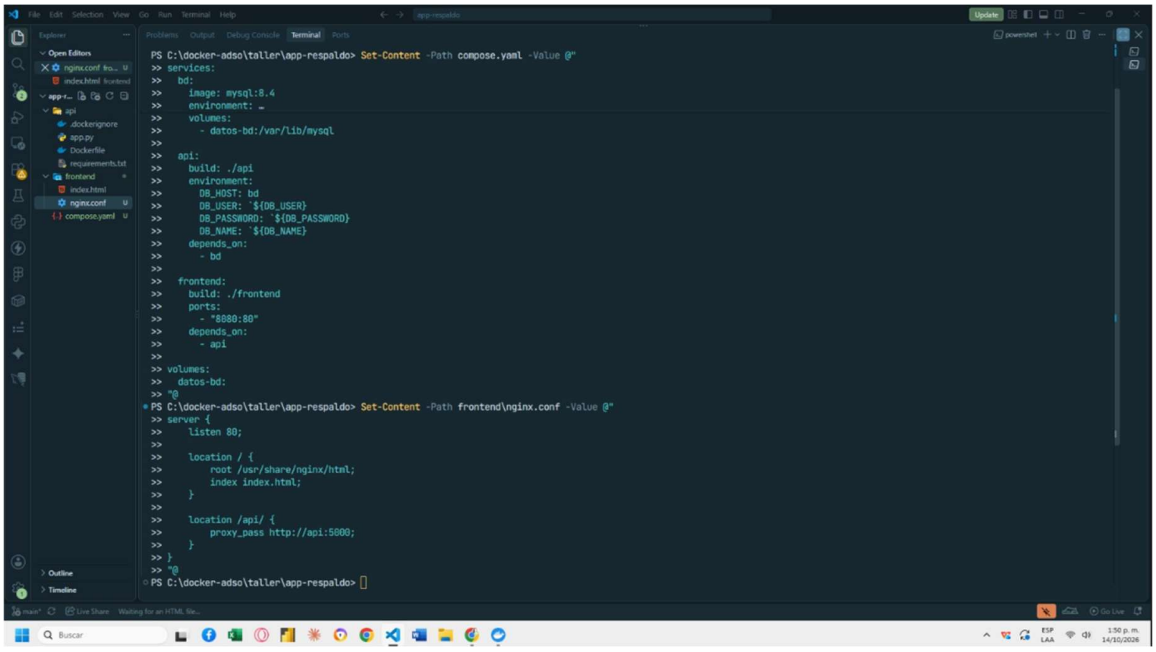


Evidencia 5 (Fase 5: Docker Compose)

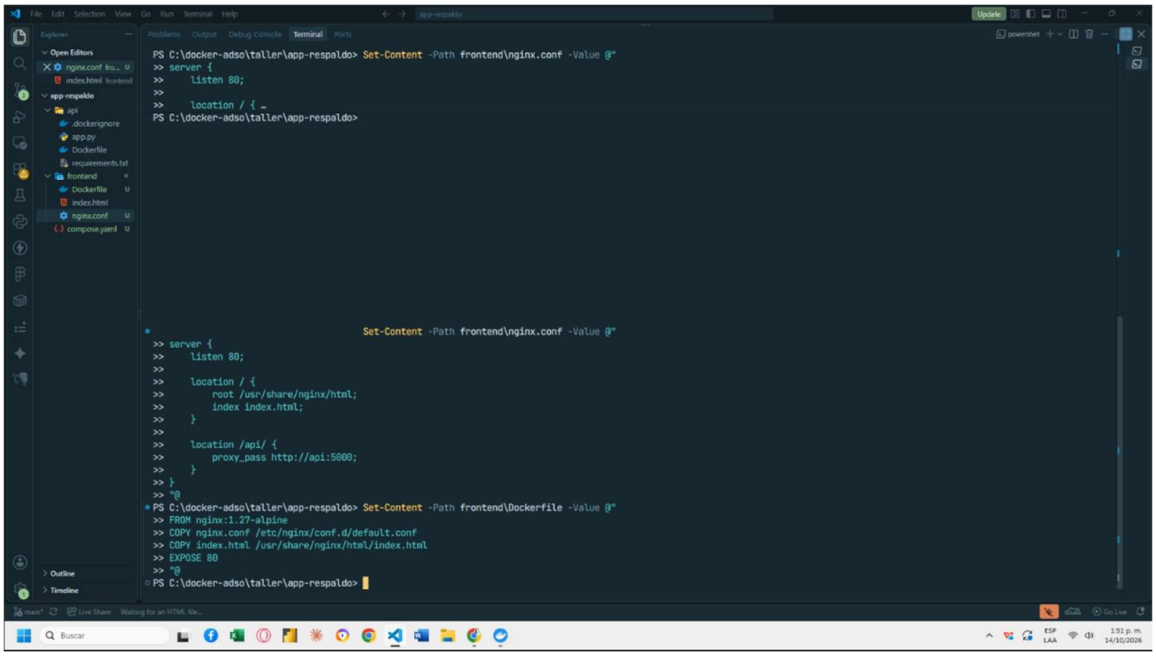
- **Cuándo la tomamos:** En el Paso 15 de la Fase 5 (el paso final del despliegue).
- **Comandos previos:** Encendimos todo con docker compose up -d --build, apagamos la infraestructura con docker compose down, volvimos a encender con docker compose up -d y verificamos su estado con docker compose ps.
- **Qué se ve en nuestra captura (son tres cosas):**
    1. Nuestra terminal mostrando la salida de docker compose ps con los tres servicios (api, bd, y frontend) en estado Up.
    2. Nuestro navegador en http://localhost:8080 mostrando la página del gestor con las tareas que agregamos inicialmente.
    3. Nuestro navegador (después de apagar y encender todo) demostrando que esas tareas siguen ahí intactas.



```
PS C:\docker-adso\taller\app-respaldo> cd C:\docker-adso\taller\app-respaldo
PS C:\docker-adso\taller\app-respaldo> Set-Content -Path compose.yaml -Value @"
>> services:
>>   bd:
>>     image: mysql:8.4
>>     environment:
>>       MYSQL_ROOT_PASSWORD: `${DB_ROOT_PASSWORD}
>>       MYSQL_DATABASE: `${DB_NAME}
>>       MYSQL_USER: `${DB_USER}
>>       MYSQL_PASSWORD: `${DB_PASSWORD}
>>     volumes:
>>       - datos-bd:/var/lib/mysql
>>
>>   api:
>>     build: ./api
>>     environment:
>>       DB_HOST: bd
>>       DB_USER: `${DB_USER}
>>       DB_PASSWORD: `${DB_PASSWORD}
>>       DB_NAME: `${DB_NAME}
>>     depends_on:
>>       - bd
>>
>>   frontend:
>>     build: ./frontend
>>     ports:
>>       - "8080:80"
>>     depends_on:
>>       - api
>>
>> volumes:
>>   datos-bd:
>> "@
PS C:\docker-adso\taller\app-respaldo>
```

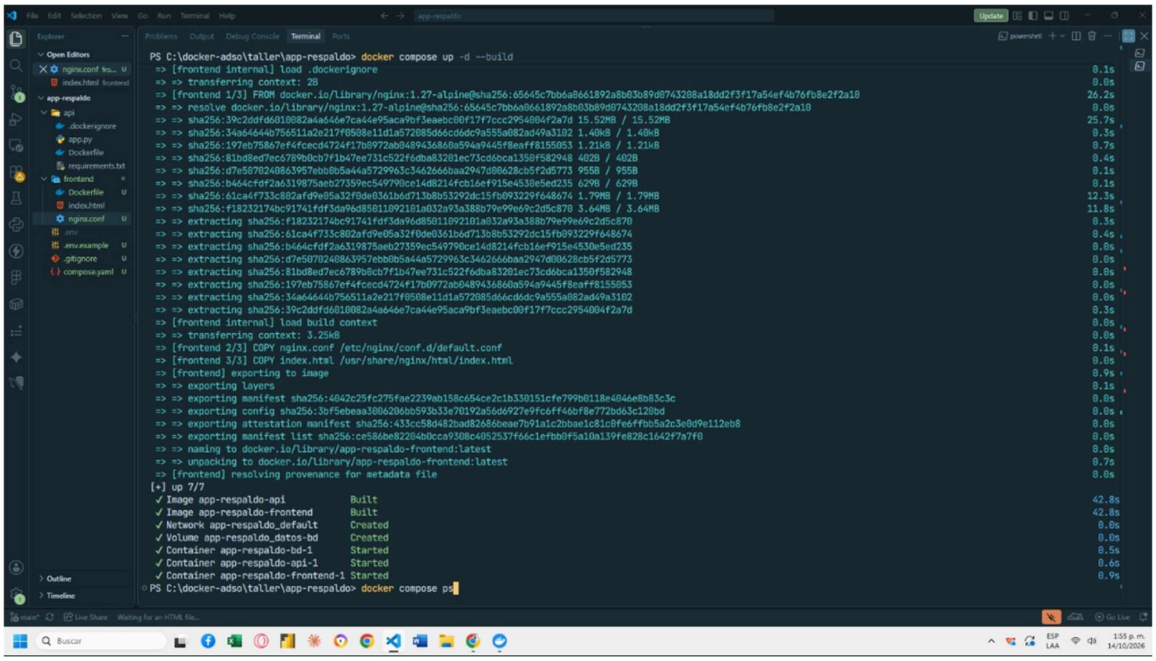


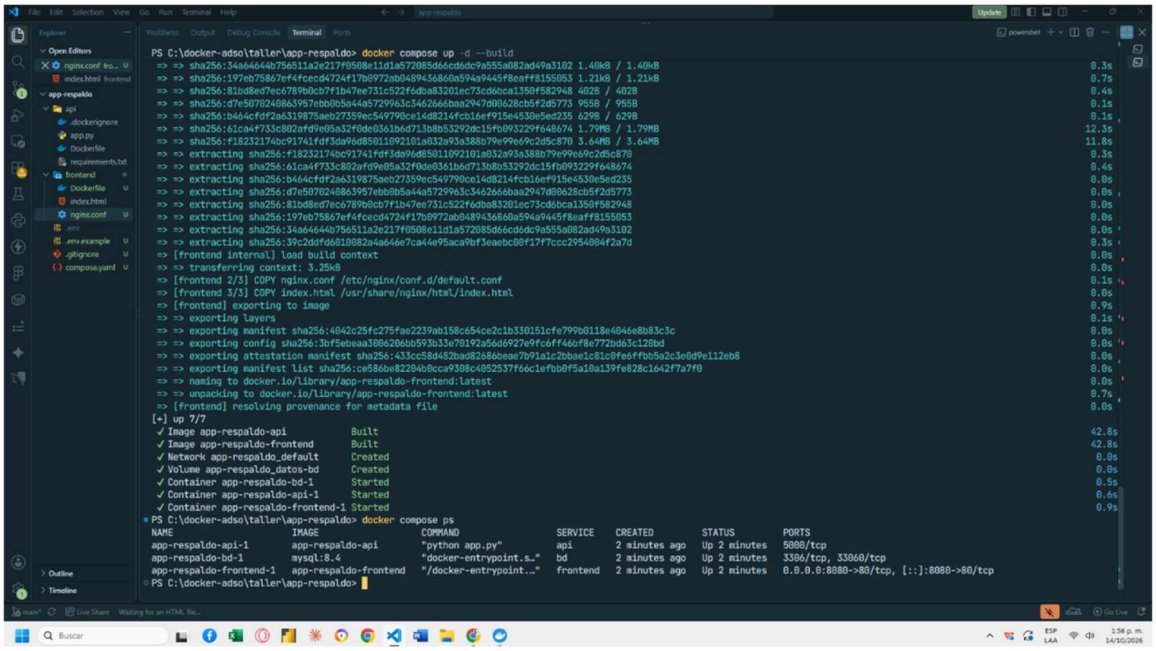
```
PS C:\docker-adso\taller\app-respaldo> Set-Content -Path compose.yaml -Value @"
>> services:
>>   bd:
>>     image: mysql:8.4
>>     environment: …
>>     volumes:
>>       - datos-bd:/var/lib/mysql
>>
>>   api:
>>     build: ./api
>>     environment:
>>       DB_HOST: bd
>>       DB_USER: `${DB_USER}
>>       DB_PASSWORD: `${DB_PASSWORD}
>>       DB_NAME: `${DB_NAME}
>>     depends_on:
>>       - bd
>>
>>   frontend:
>>     build: ./frontend
>>     ports:
>>       - "8080:80"
>>     depends_on:
>>       - api
>>
>> volumes:
>>   datos-bd:
>> "@
PS C:\docker-adso\taller\app-respaldo> Set-Content -Path frontend\nginx.conf -Value @"
>> server {
>>     listen 80;
>>
>>     location / {
>>         root /usr/share/nginx/html;
>>         index index.html;
>>     }
>>
>>     location /api/ {
>>         proxy_pass http://api:5000;
>>     }
>> }
>> "@
PS C:\docker-adso\taller\app-respaldo>
```



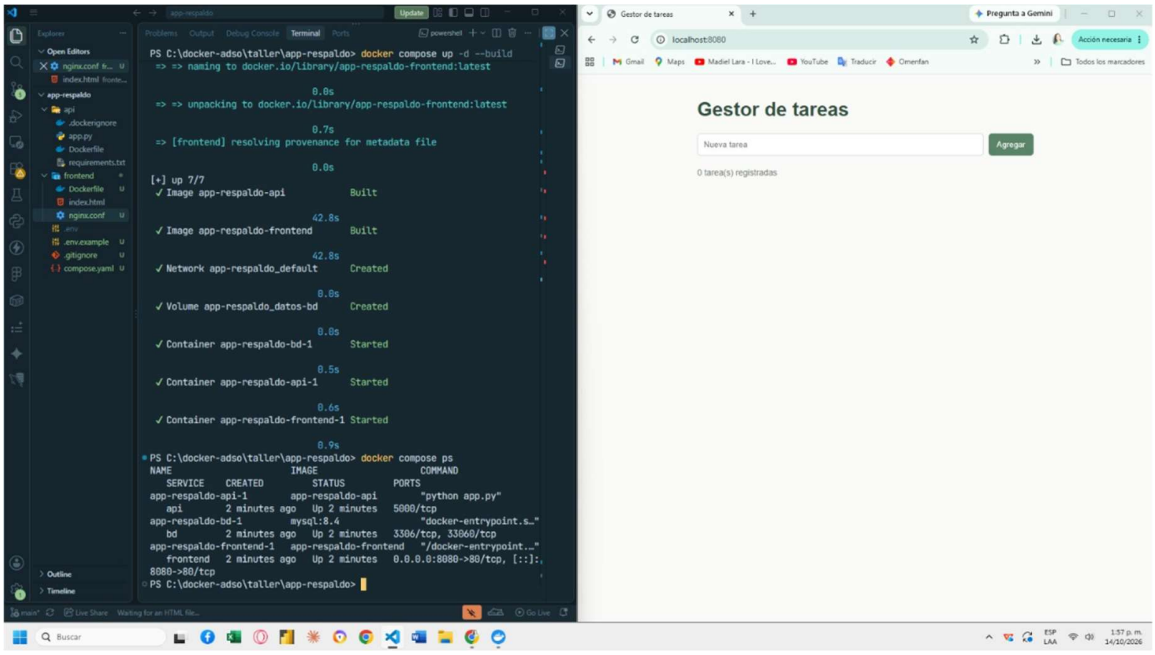
Construir y levantar los servicios

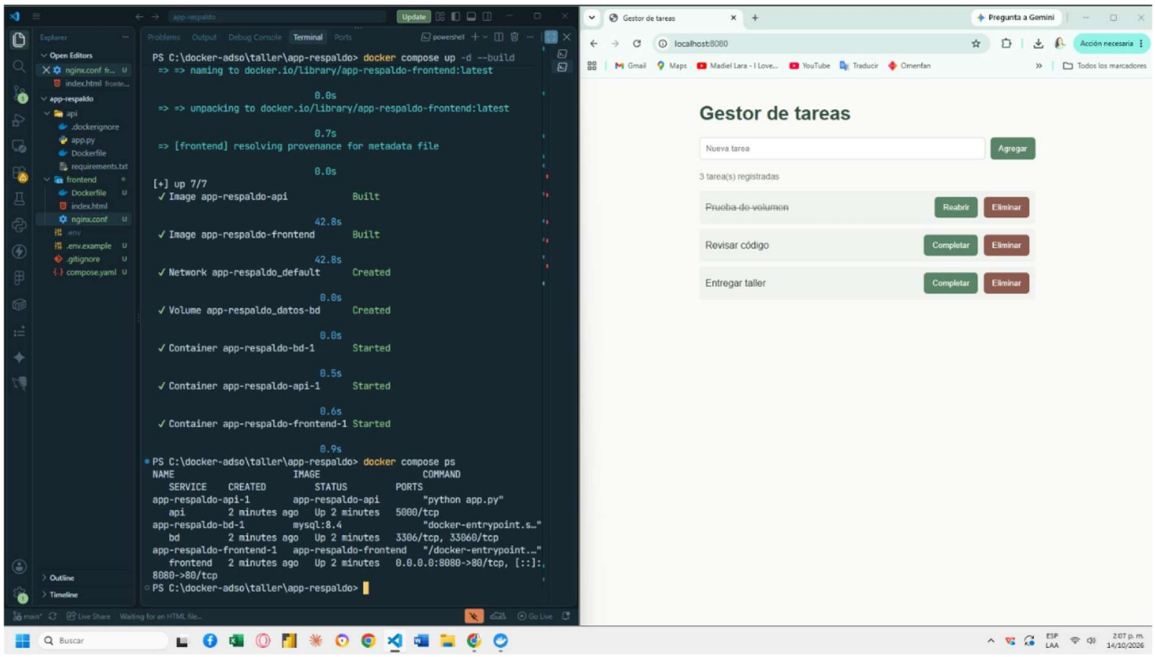
Paso 10 encender los contenedores en segundo plano y luego 2.Revisar el estado de los servicios:Paso 11.Comprobamos que todos los contenedores estén activos:



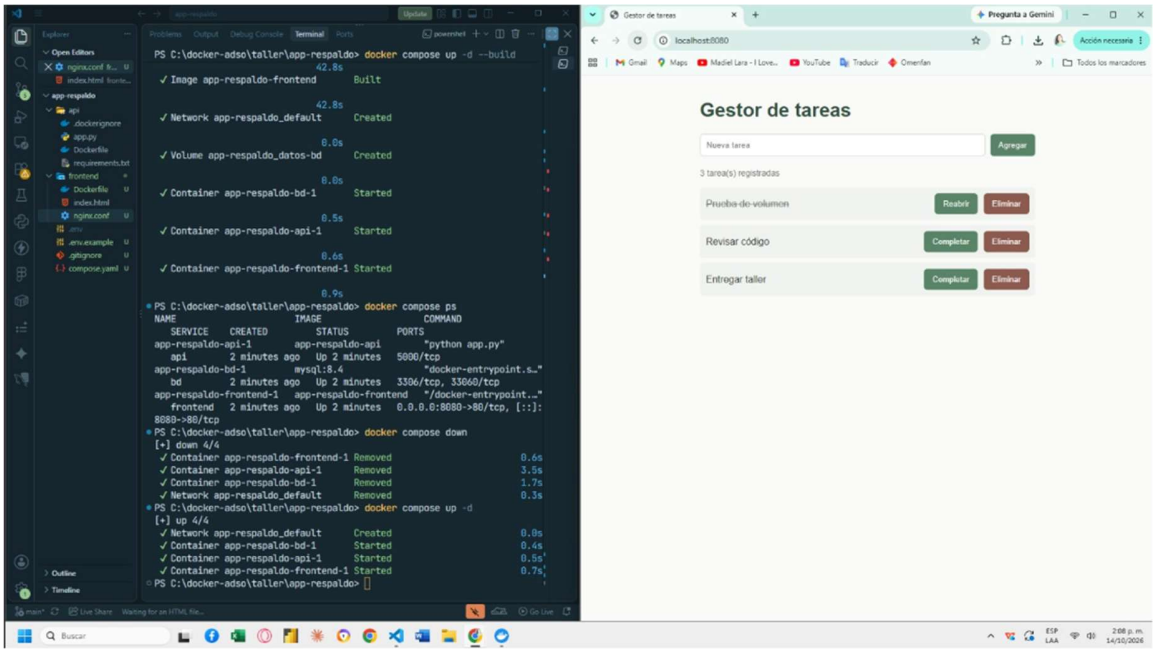


3.Probar la aplicación en el navegador:Paso 12.Esperamos unos 30 segundos mientras MySQL inicia y abre la siguiente dirección en el navegador





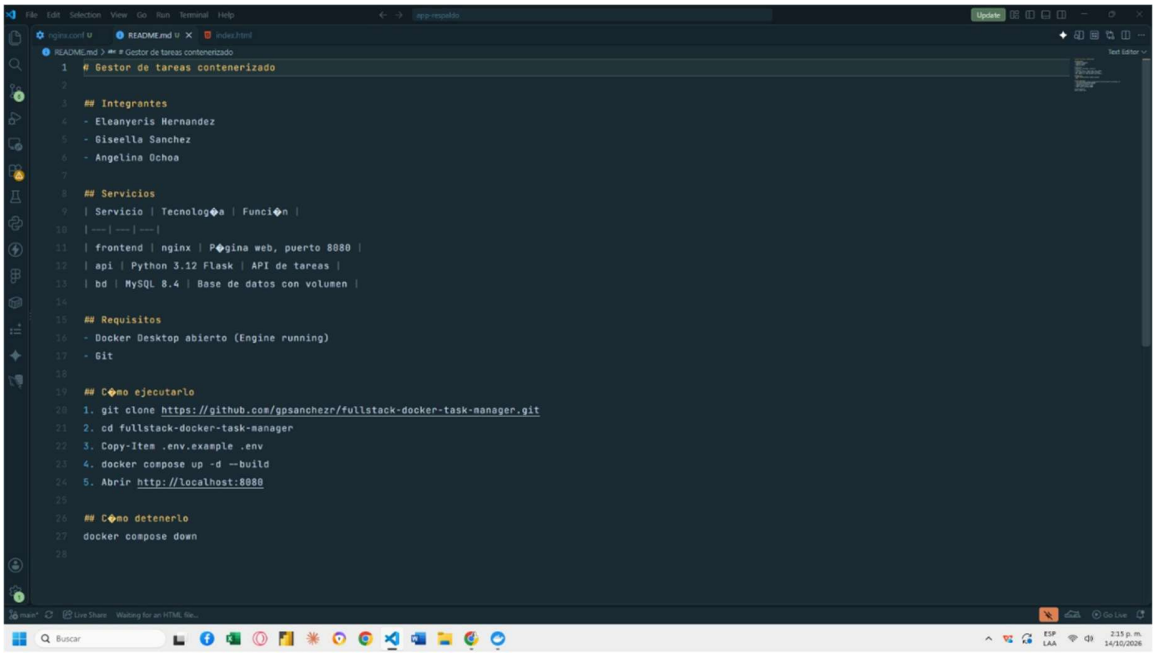
En los pasos 14 y 15 de la Fase 5.Comandos previos:Primero preparé toda la estructura de orquestación. Creé el archivo compose.yaml para unir los tres servicios, los archivos .env y .gitignore para manejar las contraseñas de forma segura, y el nginx.conf con el Dockerfile del frontend. Luego, ejecuté docker compose up -d --build para encender toda la arquitectura de un solo golpe. Verifiqué que todo estaba corriendo con docker compose ps. Fui al navegador en localhost:8080, agregué tres tareas y marqué una como completada. Finalmente, ejecuté docker compose down para apagar todo y lo volví a encender con docker compose up -d para comprobar la persistencia de los volúmenes. Qué se ve en nuestra captura: Se observa mi consola de VS Code mostrando que los contenedores se detuvieron, se eliminaron y se volvieron a crear con éxito. Al lado, está mi navegador demostrando que, a pesar de haber apagado todo, las tareas de "Prueba de volumen", "Revisar código



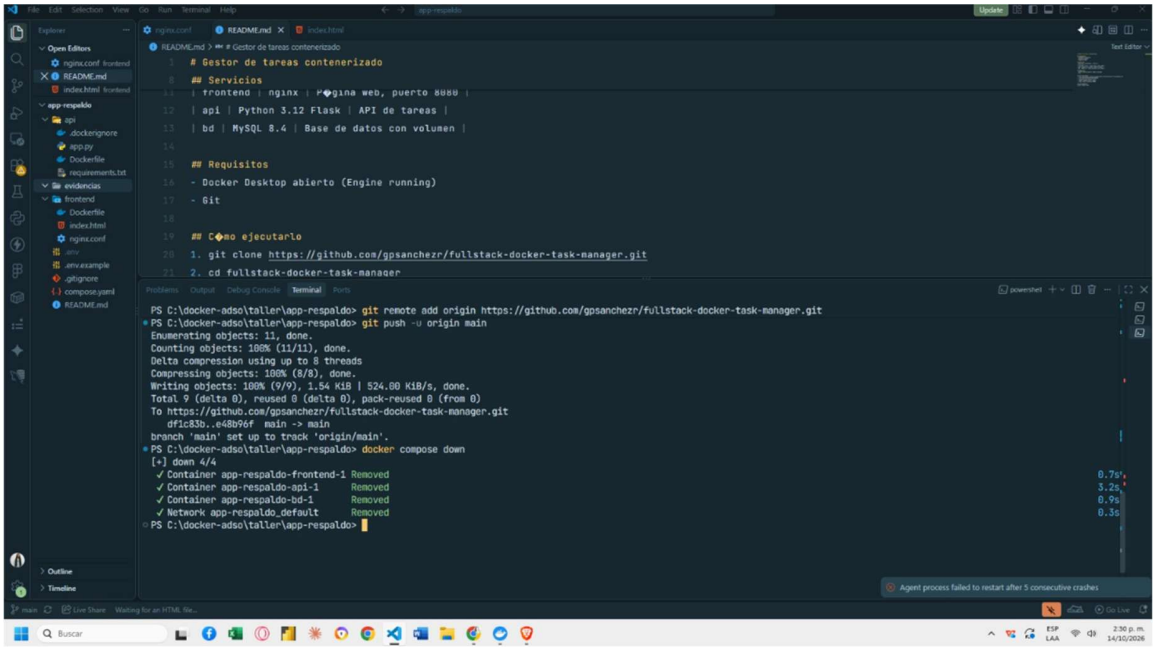
Paso final clave completado: En la Fase 6, antes de subir todo a nuestro GitHub, creamos una carpeta en nuestro proyecto llamada evidencias y guardamos allí estas 5 imágenes (nombradas evidencia1.png, evidencia2.png, etc.) para cumplir con el criterio de entrega.

README:

```
PS C:\docker-adso\taller\app-respaldo> Set-Content -Path README.md -Value @"
>> # Gestor de tareas contenerizado
>>
>> ## Integrantes
>> - Eleanyeris Hernandez
>> - Giseella Sanchez
>> - Angelina Ochoa
>>
>> ## Servicios
>> | Servicio | Tecnología | Función |
>> |---|---|---|
>> | frontend | nginx | Página web, puerto 8080 |
>> | api | Python 3.12 Flask | API de tareas |
>> | bd | MySQL 8.4 | Base de datos con volumen |
>>
>> ## Requisitos
>> - Docker Desktop abierto (Engine running)
>> - Git
>>
>> ## Cómo ejecutarlo
>> 1. git clone https://github.com/gpsanchezr/fullstack-docker-task-manager.git
>> 2. cd fullstack-docker-task-manager
>> 3. Copy-Item .env.example .env
>> 4. docker compose up -d --build
>> 5. Abrir http://localhost:8080
>>
>> ## Cómo detenerlo
>> docker compose down
>> "@
PS C:\docker-adso\taller\app-respaldo>
```



subir proyecto a repositorio apagar el proyecto:Paso 7.liberar el puerto 8080 apagando nuestros contenedores para evitar conflictos.



Evidencia 6 (Fase 6: Prueba Cruzada en otro equipo)

Cuándo la tomamos: En los pasos 8 y 9 de la Fase 6, durante la comprobación del proyecto en una computadora diferente.

Comandos previos: Nuestra compañera clonó el repositorio directamente desde GitHub ejecutando git clone https://github.com/gpsanchezr/fullstack-docker-task-manager.git. Luego, ingresó a la carpeta del proyecto, configuró las credenciales copiando el archivo .env.example a .env, y levantó toda la arquitectura ejecutando docker compose up -d --build.

Qué se ve en nuestra captura: En la parte izquierda, la consola de VS Code demuestra que la imagen se construyó y los tres contenedores (bd, api, frontend), junto con su volumen de persistencia, iniciaron correctamente en un equipo completamente distinto al original. En la parte derecha, el navegador en http://localhost:8080 confirma que el Gestor de Tareas funciona a la perfección. Se agregaron las tareas "Clonar repositorio de GitHub", "Levantar contenedores Docker" y "Redactar archivo de prueba cruzada", validando que la aplicación es 100% portable y funcional siguiendo únicamente las instrucciones de nuestro archivo README.md

